

Muzeel: Assessing the Impact of JavaScript Dead Code Elimination on Mobile Web Performance

Jesutofunmi Kupoluyi
jdk461@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

Russell Coke
russell.coke@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

Moumena Chaqfeh
moumena@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

Waleed Hashmi
waleedhashmi@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

Matteo Varvello
matteo.varvello@nokia.com
Nokia
New Jersey, USA

Lakshmi Subramanian
lakshmi@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

Yasir Zaki
yasir.zaki@nyu.edu
New York University Abu Dhabi
Abu Dhabi, UAE

ABSTRACT

To quickly create interactive web pages, developers heavily rely on (large) general-purpose JavaScript libraries. This practice bloats web pages with complex unused functions *dead code* which are unnecessarily downloaded and processed by the browser. The identification and the elimination of these functions is an open problem, which this paper tackles with MUZEEL, a black-box approach requiring neither knowledge of the code nor execution traces. While the state-of-the-art solutions stop analyzing JavaScript when the page loads, the core design principle of MUZEEL is to address the challenge of dynamically analyzing JavaScript after the page is loaded, by emulating all possible user interactions with the page, such that the *used functions* (executed when interactivity events fire) are accurately identified, whereas unused functions are filtered out and eliminated. We run MUZEEL against 15,000 popular web pages and show that half of the 300,000 JavaScript files used in these pages have at least 70% of unused functions, accounting for 55% of the files' sizes. To assess the impact of dead code elimination on Mobile Web performance, we serve 200 MUZEEL-ed pages to several Android phones and browsers, under variable network conditions. Our evaluation shows that MUZEEL can speed up page loads by 25-30% thanks to a combination of lower CPU and bandwidth usage. Most importantly, we show that such savings are achieved while maintaining the pages' visual appearance and interactive functionality.

CCS CONCEPTS

• **Information systems** → **World Wide Web**; *Information systems applications.*

KEYWORDS

JavaScript, User Experience, Web Simplification, Mobile Web

ACM Reference Format:

Jesutofunmi Kupoluyi, Moumena Chaqfeh, Matteo Varvello, Russell Coke, Waleed Hashmi, Lakshmi Subramanian, and Yasir Zaki. 2022. Muzeel: Assessing the Impact of JavaScript Dead Code Elimination on Mobile Web Performance. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC '22)*, October 25–27, 2022, Nice, France. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3517745.3561427>

1 INTRODUCTION

The reuse of existing JavaScript code is a common web development practice which speeds up the creation and amendment of web pages, but it requires sending large JavaScript files to web browsers, even when only part of their code is required. This paper aims to achieve two main objectives: 1) effectively eliminate JavaScript *dead code*, which consists of the functions that are included in web pages but never used by the browser, and 2) assess the impact of such elimination on the performance of Mobile Web.

The elimination of dead JavaScript code offers two main advantages. First, *data savings* as unused functions are not wastefully downloaded. Second, *performance savings* (CPU, memory, and battery) from avoiding useless processing at the browser [45]. Such combined data and performance savings can be significant especially for millions of users in developing regions, who access the web via low-end smartphones [13] and slow/expensive access networks [37].

The identification of dead JavaScript code remains an open problem due to the following challenges: 1) the dynamic nature of JavaScript that hinders the static analysis of its source code [38], 2) the many ways in which a user can interact with a web page, and 3) the highly dynamic changes in the structure and the state of modern web pages occurring due to the user interactivity. Previous

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IMC '22, October 25–27, 2022, Nice, France

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9259-4/22/10...\$15.00

<https://doi.org/10.1145/3517745.3561427>

works partially realized JavaScript dead code identification, because they either ignore the user interactivity [26] as they limit their dynamic analysis up to the `onLoad` event [38], or remove “less-useful” rather than “unused” functions [35] – which drastically impacts the functionality of the pages.

We address these shortcomings with MUZEEL (“eliminator” in Arabic), a novel mechanism for JavaScript code identification and elimination. MUZEEL automatically identifies – and realizes via user emulation – the order of events needed to cover a complete set of states a web page can reach at run-time. Unlike the state-of-the-art, MUZEEL does not stop the dynamic analysis of JavaScript when the page is loaded, but rather continues to emulate all possible interactions with the page, such that the functions fired due to these interactions are not mistakenly eliminated, and unused functions are accurately identified during the analysis. Apart from ownership of the JavaScript code, MUZEEL requires neither implementation details of such code nor execution traces, which set it apart from recent related works [27, 44]. It can thus be offered as a service by Content Delivery Networks (CDNs) to improve their customer experience.

We use MUZEEL to quantify the amount of dead JavaScript code in the web. We consider 15,000 popular web pages – as per Alexa list [17] – to find that half of the 300,000 JavaScript files host 70% of unused functions, accounting for about 55% of their files’ sizes. Our analysis further shows that 85% of the web pages can be “Muzeel-ed” in less than 5 minutes by a modern server. This implies that a CDN can easily run MUZEEL each time a page is updated to ensure the correctness of the dead code elimination.

To assess the impact of dead code elimination, we evaluate: 1) MUZEEL’s performance across several Android phones and browsers (Chrome, Firefox, Edge, Brave), and 2) the visual appearance and interactive functionality of MUZEEL-ed pages when compared to the original versions. Results show significant performance improvements, e.g., up to 9 seconds faster pages on both low-end and high-end phones when on 3G. These speedups derive from a combination of lighter pages (median reduction of about 400KB) and lower CPU usage. Additionally, MUZEEL maintains the appearance and the interactive functionality for 90% of the pages. In summary, the contributions of this paper are:

- Analyzing JavaScript in 15,000 popular web pages to investigate the percentage of unused functions and the potential of eliminating these functions on page size reduction. Our analysis shows that 70% of JavaScript functions on the median page are unused, and the elimination of these functions would contribute to the reduction of the page size by 55%.
- Proposing *MUZEEL*; a novel solution for eliminating unused JavaScript functions in web pages through a complete user interactivity emulation process that fully considers all possible ways to interact with the pages.
- Evaluating the impact of MUZEEL on mobile page performance and quality using several Android mobile phones and browsers, showing significant speedups in page loads and speed index, and a reduction in page size by around 0.4 Megabyte, while maintaining the pages’ visual appearance and interactive functionality.

2 RELATED WORK

Several solutions exist to mitigate JavaScript cost. *Uglifiers* (also known as minifiers) [18, 24] are popular tools which reduce the size of JavaScript files by removing unnecessary characters such as new lines, white spaces, and comments. Uglifiers offer data savings deriving from smaller JavaScript files; however, the browser still has to interpret the entire JavaScript code which requires a similar processing time as in the case of the original code.

Script-streaming is a JavaScript optimization introduced in Chrome v41. With script-streaming, large JavaScript files are parsed as they download offering 10% faster pages, according to Google [31]. Script-streaming is a client-side solution that existing web pages can already take advantage of. However, there is no guarantee on which JavaScript files embedded on a web page are being “script-streamed” as this depends on the browser support and the availability of the streaming thread [15]. Unlike both uglifiers and script-streaming, MUZEEL mitigates the cost of JavaScript in web pages by eliminating unused code. Thus, it can be used in conjunction with both to offer additional data and processing savings.

To the best of our knowledge, Lighthouse is the main tool offering insights on potential JavaScript (and CSS) dead code detection [32]. However, Lighthouse does not consider the user interactions in the decision process, and only informs the developer that a JavaScript code isn’t needed for page load encouraging “code splitting”, *i.e.*, separating the code needed for the initial page from the code that might be needed later.

Lacuna [38] is the state-of-the-art approach which focuses on the elimination of JavaScript dead code at the functional level using dynamic analysis. Lacuna’s main contribution is the development of a generic framework where existing JavaScript analysis techniques can be applied in combination. With respect to dynamic JavaScript analysis, Lacuna uses a simple mechanism which loads a page in PhantomJS, one of the first headless browsers [39], to identify and eliminate the unused functions. It stops the analysis once the page is loaded, ignoring the functions that might be needed when the user interactivity events are fired. These functions would be incorrectly identified as being unused, since they are not required during the page load. In comparison, MUZEEL aims at automatically identifying all user events (and their order) needed to cover all possible states a web page can reach at run-time, so that unused functions can be accurately identified. As demonstrated in our evaluation, MUZEEL significantly improves the identification of unused JavaScript functions in comparison to Lacuna, proved by the production of pages that are mostly equivalent to the original pages in terms of both their visual appearance and interactive functionality.

Another two recent works [27, 44] have explored JavaScript dead code elimination via dynamic analysis. [44] relies on the unit tests of a JavaScript application to generate execution traces, while [27] requires monitoring user interaction with pages, e.g., via real user monitoring systems (RUM) [16]. MUZEEL improves upon both approaches since it neither requires unit tests that are usually unavailable for web pages, nor user monitoring, which is time-consuming, prone to errors, and privacy-invasive. We did not perform a head-to-head comparison with [44] or [27] since their code is not available,

and their requirements do not scale to broad applicability to the Web (or the 15,000 web pages we studied with MUZEEL).

Recently, the authors of [35] propose to remove *less-useful* functions from the JavaScript code of web pages to minimize memory usage in low-end devices. They represent a given web page as a set of components structured as a dependency tree and explore cutting in a bottom-up fashion. The paper shows that, on average, a 40% loss of page functionality can offer 50% memory savings. MUZEEL is orthogonal to [35] since it aims at fully maintaining the page functionality.

The key impact of JavaScript on Page Load Time is shown in [45], which is mainly occurring due to its role in blocking the page rendering. With this in mind, the state-of-the-art web acceleration solutions suggest to restructure the page loading process [46], use the pages' dependency structure for prioritizing critical resources' requests [36], or provide server-based mechanisms as in Vroom [40] to accurately aid clients in discovering web resources, so that each resource is fetched directly from its hosting domain by pushing the content of dependent resources and returning dependency hints to clients. These hints inform clients about the dependent resources that should be fetched from other domains, so that they don't incur the overhead of fetching resources unnecessarily (many of these resources will be already in the client's cache by the time of discovering the need for them). Server-side pushes and client-side fetches are coordinated in Vroom such that the resources requiring to be parsed or executed (mainly JavaScript) are received earlier than the others. This way, resources arrive at the client in the order in which they will be processed. With most of the state-of-the-art web acceleration solutions, the browser has to download the full JavaScript resources brought by a given web page and process them.

Recently, a number of solutions focused on accelerating the page load by removing JavaScript elements that are non-critical to the page content or interactive functionality. The core issue with such solutions is how to distinguish between elements that are critical and the ones that are not while having a minimal impact on the final page content and functionality. In [22] the classification of JavaScript elements, into critical and non-critical, is facilitated via a rule-based method, while in [21], the use of machine learning is proposed to improve the classification accuracy. However, the dataset created to evaluate [21] relies on third-party JavaScript libraries, resulting in a classification accuracy not exceeding 90%. To create a larger representative dataset, the authors of [20] proposed a tool for developers such that a large number of JavaScript elements can be annotated into critical and non-critical, by assessing the impact of non-critical elements to be removed in order to generate lighter versions of existing pages. In contrast to the aforementioned solutions, MUZEEL aims at optimizing the code of JavaScript elements rather than completely removing them. In essence, MUZEEL can be used in combination with the previous solutions to further optimize the critical JavaScript elements by removing their dead code.

3 MOTIVATION AND CHALLENGES

The identification and elimination of JavaScript dead code from modern web pages is challenging and remains an open problem to date. In the following, we briefly discuss these challenges.

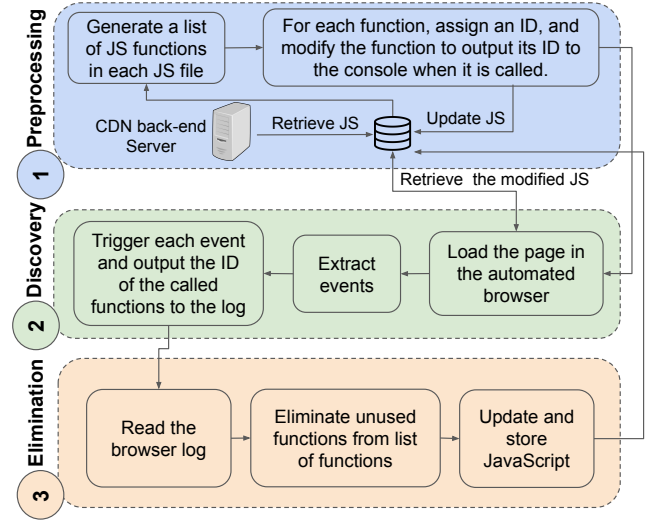


Figure 1: MUZEEL's architecture.

The Dynamic Nature of JavaScript. JavaScript's dynamicity hinders static analysis of its source code. More specifically, language features such as the possibility of dynamically accessing objects' properties, and context binding that allows developers to assign an arbitrary object to "this" keyword, require a run-time analysis of the code to identify unused functions.

Handling User Interactivity. User interactivity is a key feature that JavaScript provides to web pages. Accurate JavaScript dead code identification needs to be paired with user activities like scrolling the page or entering a menu. Different run-time execution flows are possible due to unpredictable user interactions with the page. A major challenge to address by a dynamic web-based JavaScript analyzer is to handle all combinations of interactive events in a given web page. MUZEEL achieves this by constructing an accurate representation of the page elements along with their associated events, which requires the discovery and consideration of events dependencies across the page.

Handling Highly-Dynamic Pages. The dynamism of web pages adds another level of complexity to the aforementioned challenge. More specifically, interactive elements can be added dynamically to the page, and existing elements can be modified as the user interacts with the page. With this in mind, potential changes in the page structure and state are required to ensure all events are identified.

4 MUZEEL

This section describes MUZEEL, a dynamic JavaScript analyzer that autonomously discovers and eliminates dead-code in web pages. MUZEEL relies on the identification of JavaScript functions associated with *page-load events* – or required to load the page such as *onLoad* – and *user events* that occur due to user interactivity with the page. User and page-load events are normally associated with "event handlers" which listen to events and call JavaScript functions when events fire. The JavaScript functions that handle the page-load events are easily identified since they are reached via a

single path consisting of a sequence of events occurring from when the page is requested until being loaded. MUZEEL addresses the challenge of identifying the JavaScript functions triggered by user events through emulating all possible paths of user interactivity with a given page and reporting *unused* functions.

MUZEEL can be used by web page owners to optimize JavaScript code within their pages, or *first-party* JavaScript. It can also operate on *third-party* open-source/copyleft JavaScript files (such as `Chartist.js` [25]) granted that the web page owner agrees to host local versions of these files. This is needed so that a Muzeel-ed version of a JavaScript file can be generated; alternatively, the third-party would have to generate Muzeel-ed versions of such file for each web page and, upon request, figure out which version of the file to serve. While this approach would allow MUZEEL to operate also on copyrighted third-party JavaScript code, we have dismissed it due to its poor scalability (maintain one JavaScript version per requesting website) and complexity in identifying the originating request, e.g., due to the poor reliability of the HTTP referrer field [33].

Apart from ownership of the JavaScript code, MUZEEL requires neither implementation details of such code nor execution traces [27, 44]. As such, it can be offered as a service by CDN providers to improve their customers' experiences. Figure 1 shows MUZEEL's architecture with its main operations: pre-processing, dead code discovery, and dead code elimination. In the remainder of this section, we discuss each of these operations in detail.

4.1 Pre-Processing

At this phase, JavaScript functions (across multiple files) are instrumented to `console.log()` [34] a unique "function ID" (`<file_name, function_startLine, function_endLine>`) to the browser console when the function is called by any event. Function IDs that are not logged to the browser's console after the dead code discovery operation identify *unused functions* or *dead code*.

4.2 Dead Code Discovery

To discover dead code, MUZEEL loads a web page in an automated browser, waits for a predefined period of time after the page is loaded to give the chance for the download and execution of the asynchronous JavaScript, and then emulates the user interactivity events. During this time (page load time plus the waiting time afterward, and the time required for emulation), JavaScript functions that handle the page events (including the page load events and the user interactivity events) are logged to the browser console. A user event is associated with a *page element*, defined as an object that appears in an HTML tag within the Document Object Model (DOM) of the loaded page (such as an image, a button, or a navigation element). For a given web page, MUZEEL identifies all page elements appearing in the DOM structure of the page after loading it in the automated browser and waiting for a predefined period of time (see Section 4.2.1), and extracts the set of all user events (such as click or mouseover) associated with each of them to drive the emulation of user interactivity (see Section 4.2.2).

4.2.1 Page Elements Identification. MUZEEL uses XPath to uniquely identify page elements across reloads. XPath [4] is a query language adopted by several browsers and browser automation tools, and is

conventionally constructed using two strategies [4]: attribute-based and position-based. The attribute-based strategy identifies elements using their attributes, e.g., the "id" attribute or the "class" attribute. This strategy may result in duplicate XPath since developers might mistakenly use the same id for multiple elements, and a given class can be applied to multiple elements. The position-based strategy returns an element at a given position from a certain tag, e.g., `//p[1]` identifies the first paragraph in a web page. This strategy also fails to uniquely identify elements across reloads, in situations where elements are dynamically added/removed from the page (e.g., user interaction with the page).

To overcome these limitations – and to uniquely identify elements in a given web page across reloads – MUZEEL combines both approaches in a hybrid method which uses position-based indexing from the nearest parent element for which the attribute-based strategy can be used. MUZEEL uses the id or the class attribute alongside an index to identify an element, e.g., `(//a[@class='anchor'])[1]` identifies the first "a" tag on the page, with a class attribute of "anchor". Elements with no id or class attribute are accessed using the position from the nearest element with an id or class, and if none exists, they are accessed from the body tag. As mentioned earlier, MUZEEL considers all page elements appearing in the DOM structure of the page when loaded in the browser. It waits for additional period of time before identifying those elements to allow dynamic DOM changes that might occur due to the execution of asynchronous JavaScript. The DOM elements that are added later (either due to the execution of asynchronous JavaScript or due to user interactivity) are currently not considered by MUZEEL.

In the following, we provide a clarifying example using a sample page (see Listing 1). The first "`<div>`" element has an "id" attribute (L3), so the XPath is derived as follows: `(//div[@id = "div1"])[1]`. Next, the XPath for the "`<button>`" element (L4) is identified as: `(//div[@id = "div1"])[1]/button[1]`. The second "`<div>`" element has a "class" attribute (L6) and it is represented as: `(//div[@class = "divClass2"])[1]`. Lastly, although the "`<a>`" element (L7) contains a "style" attribute, which is not used by MUZEEL as an identifying attribute, and the "`<a>`" element is identified from "divClass2" as `(//div[@class = "divClass2"])[1]/a[1]`.

```

1  <html>
2    <body>
3      <div id="div1">
4        <button></button>
5      </div>
6      <div class="divClass2">
7        <a style="color:blue;"></a>
8      </div>
9    </body>
10 </html>

```

Listing 1: A sample page demonstrating XPath construction

4.2.2 Emulating User Interactivity. MUZEEL emulates user interactivity on a web page to trigger the user events associated with the identified DOM elements. Using a browser automation tool, common user events are generated, including but not limited to: *mousedown*, *mouseup*, *mouseover*, *mouseout*, *keydown*, *keypress*, *keyup*, *dblclick*, *drag*, *dragstart*, and *dragend*. Due to events dependency, some events are only successful if triggered after the occurrence of another set of events. For example, the elements under the "More"

Algorithm 1 Muzeel User Interactivity Emulation

```

1: INPUT : xpathList[]
2: OUTPUT : baseEvent
3: procedure TRIGGEREVENTS(xpathList)
4:   List<Event> eventList;
5:   for xpath in xpathList do
6:     xpathEvents = getEventsFromXpath(xpath);
7:     for event in xpathEvents do
8:       eventList.add(event);
9:   Event baseEvent;
10:  Queue<Event> eventQueue;
11:  eventQueue.add(baseEvent);
12:  while eventQueue ≠ ∅ do
13:    parentEvent = eventQueue.pop();
14:    refreshPage();
15:    triggerEvent(parentEvent);
16:    ▷ predecessorEvents are triggered first
17:    for event in eventList do
18:      if triggerEvent(event) == True then
19:        parentEvent.addChild(event)
20:        ▷ predecessorEvents are assigned
21:        eventQueue.add(event)
22:        eventList.remove(event)
23:        refreshPage();
24:        triggerEvent(parentEvent)
25:  return baseEvent

```

drop-down menu in Figure 2 are only interactive after clicking “More”. In addition, triggering a certain event may prevent the interactivity with another event, such as an event that causes a “modal” window (or lightbox) to open on a page where the modal window blocks the elements appearing behind it and prevents the interactivity with these elements. An event is considered to be successfully triggered when no errors are generated upon the execution of the function that handles it.

MUZEEL considers both event dependency and page state changes when emulating user interactivity. Algorithm 1 provides the details of how user interactivity emulation works in MUZEEL. First, the list of “Event” (*eventList*) on the page is obtained (L4-8). Each “Event” is an object with the following attributes: the *eventType* (which refers to the type of event, e.g., *click* or *keydown*, etc.), the *XPath* of the element the event is associated with, the parent event (*parentEvent*) that is required to be triggered before the event can occur, and a list of dependent or successor events *successorEvents*. When MUZEEL gets the event from the *XPath*, the *parentEvent* is yet to be determined. MUZEEL uses breadth-first search to identify the parent event for every event in the *eventList*. It instantiates an *eventQueue* with a *baseEvent*, which is an ancestor of all events. When the *baseEvent* is triggered, no action is performed. While the *eventQueue* is not empty (L12), the first event from the *eventQueue* is popped and stored in the *parentEvent*. MUZEEL seeks to find all elements in *eventList* whose parent is *parentEvent* by triggering the *parentEvent*, and determining which other events can be successfully triggered after the *parentEvent*. Once an event is triggered, it is removed from *eventList* and added to *eventQueue*.

The first *parentEvent* to consider is the *baseEvent*, which results in no changes in the page state when triggered. The *successorEvents* of *baseEvent* are those events that can be directly triggered on the

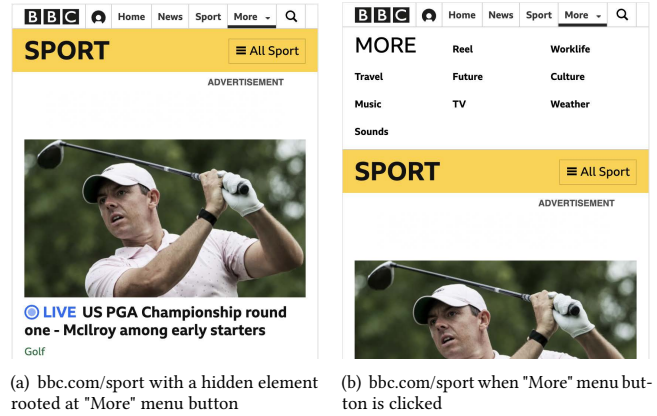


Figure 2: An example showing a hidden page element.

page load. Note that before triggering an event, MUZEEL triggers all its predecessor events. To determine *successorEvents* of *baseEvent*, MUZEEL loops through all the events in the *eventList* to find events that can be triggered from the current page state. Any event that can be triggered will be added in the *successorEvents* list of *baseEvent*. In doing so, the *baseEvent* is assigned as the *parentEvent* of the added event. The event is then removed from the *eventList* so it is not triggered again by *baseEvent*. It is then added to *eventQueue* so that in a subsequent iteration of the while loop (L12), we can find all events which have this added event as their *parentEvent*. After this is done, we refresh the page and trigger the *baseEvent* to return to the page state of *baseEvent* (a fresh page load). This is required to overcome the aforementioned challenge of events blocking other events and to ensure that an accurate event dependency is determined. A full event-dependency graph is drawn from the *baseEvent* returned by this process.

4.2.3 Additional Challenges. Web pages may contain “hidden elements” that are not visible or interactive on the first page load and would only appear upon the occurrence of a certain event. For example, the navigation elements of a drop-down menu may not be visible until the menu button is clicked or hovered over. MUZEEL addresses the issue of hidden elements in web pages, by considering the events dependency such that the necessary predecessor events are triggered in the right order to reveal a hidden element and trigger any successor event afterwards (Section 4.2.2).

Interactive menus are a common feature of web pages. These elements can reveal/hide components when clicked, depending on their current state. Therefore, opening and closing can change the page state, such that the interactivity state of the page elements when a given component is opened differs from their state when that component is closed. To handle this issue, MUZEEL triggers all click events three times, where the 1st click captures the function(s) behind opening the hidden component, the 2nd click captures the function(s) behind closing the hidden component, and the 3rd click reopens the hidden component so that the user interactivity with the successor elements included in the component can be emulated. If the same function handles opening/closing, then it will be logged twice, however, MUZEEL discards duplicate log statements.

4.3 Dead Code Elimination

When the dead code discovery process completes, MUZEEL uses the browser’s console logs – which contains the IDs of the JavaScript functions that were called – to annotate the *used* functions. The functions that are never called can be removed from their respective JavaScript files.

It is worth mentioning that MUZEEL implicitly considers nested functions. Specifically, when triggering an event leads to calling a nested function, the latter function is also called, and both are reported to the browser’s console. Similarly, in a case where a function is removed, all its subsequently nested functions are also removed. This means that even though MUZEEL does not preserve the hierarchy among the functions in the initial constructed list, it successfully obtains a complete trace of the called functions, and therefore, the functions that are not called can be accurately identified as *unused functions*.

5 IMPLEMENTATION

5.1 CDN Emulation

Given that an actual CDN deployment is challenging, we implement the next most realistic scenario for the purpose of evaluation¹. We assume a medium-large CDN provider [7] hosting the first 15,000 web pages in Alexa’s top 1M list. To do so, we dedicate a powerful server machine (64 cores and 1 TB of RAM) to crawl these 15,000 web pages. For each page, two copies of each JavaScript files are stored: *original* and *MUZEEL-ed*. The MUZEEL-ed version of a JavaScript file is updated at two points in time: 1) during pre-processing (when the function log calls are added), and 2) during dead code elimination, when the unused functions are removed.

Next, we emulate a CDN *edge* using *mitmproxy* [12] to intercept regular browser traffic and serve it from a local cache. This allows us to support unmodified testing devices, granted that they install *mitmproxy*’s root CA (Certificate Authority) to handle HTTPS traffic. Note that this step is only required for testing purposes as an actual CDN owns the certificates for the domains it serves. We assume all traffic is handled using HTTP/2 [3].

5.2 Browser Automation

MUZEEL uses Selenium [30] for browser automation. Selenium is a framework for automated web testing which allows to load web pages through different browsers, simulate and control the user-interactions with the page, access the different page elements and events, and modify the page DOM structure. MUZEEL uses the Selenium Chrome web-driver; Chrome is chosen because of its popularity as well as its accompanying DevTools Protocol (CDP) that is vital in obtaining page events and mapping them to their corresponding page elements. MUZEEL loads a web page in the automated browser, and extracts a list of page elements from the DOM structure generated by the browser along with the event listeners [11] associated with the page elements. For every element, MUZEEL generates an *XPath* identifier to locate each element as described in Section 4.2.1, identify elements across reloads, and facilitate the emulation of the user interactivity with these elements.

¹MUZEEL is open-source and its code-base is publicly available under <https://github.com/comnetsAD/Muzeel>

5.3 Identifying Elements and Extracting Events

When MUZEEL loads a web page via Selenium, it extracts the HTML DOM structure of the page rendered by the Chrome browser. It then uses *jsoup* [2] – a popular open source HTML parser – to parse the page DOM and construct the *XPaths* of the page elements (see Section 4.2.1) from a Depth First Search (DFS) traversal.

MUZEEL uses the CDP to extract the events associated with a given *XPath*. This is achieved by first obtaining the “objectId” associated with the *XPath* – the objectId is Chrome’s internal id for elements on the page. MUZEEL obtains the objectId using the Chrome Dev Tools, `document.evaluate` call. MUZEEL then passes the obtained objectId to Dev Tools `DomDebugger.getEventListener` functionality to extract the eventListener associated with the given objectId. This process is carried out for all the elements on the page. A list of events associated with an element is returned from this call. MUZEEL takes each event returned and creates an Event object for each with the *XPath* and eventType.

5.4 Emulating User Interactivity via a Bot

To emulate user events, MUZEEL implements an *Interaction Bot* that leverages Selenium bundled with Chrome Web-driver. MUZEEL navigates through the events retrieved (see Section 5.3), taking the events dependency into account (as described in Section 4.2.2). For each event, MUZEEL locates the associated element using the *XPath*, and then attempts to trigger the event on that element. It relies on the native implementation to trigger the event in case the event behavior is implemented in Selenium (such as the case of *click*, *dblclick*, and *drag*). In cases where a specific event handling is not directly supported in Selenium, that event is then handled by combining other supported user-interaction events to emulate the unsupported event. For example, to emulate a user moving in and out of the element as specified by a *mouseout* event, Selenium’s *moveToElement(element)* function is first called (moving the cursor to the element’s position), then, *moveByOffset(x,y)* is called (which moves the cursor some arbitrary offset away from where the element is located). Events that are not related to user interactions are not explicitly triggered—such as the “load” event that is triggered when page load is completed. This is because these events are triggered implicitly by the browser, thus they do not need to be explicitly triggered for their associated functions to be logged.

6 MUZEEL EVALUATION

Given the CDN deployment presented earlier, we perform an extensive evaluation of MUZEEL revolving around: 1) dead code elimination potential in the Web (15,000 pages), 2) the accuracy of the web page produced, 3) the processing cost when adopted by a CDN, 4) the impact on the end-user experience (page timing metrics, page size, and resource utilization), and 5) the interplay with different browsers, phone types, and networking conditions.

6.1 Methodology

6.1.1 Datasets. Using a powerful server machine (equipped with 64 cores and 1 TB of RAM), we “crawl” the landing pages of Alexa’s 15,000 most popular web pages. The pages are loaded via Chrome while recording the full HTTP(S) content and headers using *mitmproxy*.

Specs	Redmi Go	J3	S10
Vendor	Xiaomi	Samsung	Samsung
OS	Android 8.1 Oreo (Go edition)	Android 8.0.0 Cortex-A53	Android 9.0 (Pie) Cortex-A53
Num. CPUs	Quad-core	Quad-core	Octa-core
CPU Freq.	1.4	1.4	2x2.73, 2x2.31, 4x1.95
Memory	1GB RAM	2GB LPDDR3	8GB RAM
Battery	Li-Ion 3000mAh	2,600mAh	Li-Ion 3400mAh

Table 1: MUZEEL test-bed composition

Next, we produce MUZEEL-ed pages by running our dead code discovery and elimination mechanism. We use the full set of 15,000 cloned web pages for benchmarking MUZEEL (see Section 6.2)

From these 15,000 pages, we have then selected 200 web pages (both original and MUZEEL-ed) which we use for performance evaluation across multiple phones, browsers, and network conditions (see Section 6.3). These pages are selected as follows. First, we consider the 1,500 most popular pages from the full data-set. Next, we divide these pages into four buckets each containing a double number of webpages (100, 200, 400, and 800). Finally, from each bucket we randomly choose 50 pages.

6.1.2 Client-side Test-bed. The client-side test-bed consists of several Android phones – from low end (Xiaomi Redmi Go, Samsung J3) to high end (Samsung S10) – whose characteristics are summarized in Table 1. For most tests we rely on Chrome, as it is today’s most popular browser. We also experiment with Firefox, Edge, and Brave, which was chosen for its rising popularity [9], and advanced ad-blocker [6]. The phones connect to the Internet over a fast access with a symmetric upload/download bandwidth of about 100 Mbps. Network throttling was also used to emulate the access networks summarized below.

- **3G:** a slow cellular network with a download rate of 1.6 Mbps, upload rate of 768 Kbps, and a round trip time (RTT) to the CDN edge of 300 ms [48].
- **LTE:** a moderate cellular network with a download/upload rates of 12 Mbps, and RTT to the CDN edge of 70 ms [48].
- **LTE+:** a fast cellular network with a download rate of 42 Mbps, upload rate of 10 Mbps, and RTT to the CDN edge of 40 ms (i.e., 4G LTE-Advanced based on typical real world network speeds measured on UK’s mobile networks [14, 19]).

6.1.3 Browser Automation and Performance Metrics. Each mobile device connects via USB to a Linux machine running the WebPageTest browser automation tool. This tool is used to automate both web page loads and telemetry collection, e.g., performance metrics and network requests. We focus on classic web performance metrics [41] (FirstContentfulPaint, SpeedIndex and PageLoadTime), as well as CPU, and bandwidth usage. For the Samsung J3, we also report on battery consumption measured by a power meter directly connected to the device in battery bypass [42]. Given that not all browsers on Android allow communication with their developer tools, which is used by WebPageTest, we have also developed a tool which uses the Android Debugging Bridge (adb) [8] to automate a browser, i.e., launch and load a webpage, while monitoring resource utilization. We then leverage visualmetrics [10] to extract performance timing metrics from a video recording of the web page loading.

6.2 Muzeel Benchmarking

We now benchmark MUZEEL’s ability to correctly identify and eliminate JavaScript dead code. Unless otherwise noted, this analysis refers to the top 15,000 webpages from Alexa’s top 1M list. We consider several metrics: the numbers of eliminated JavaScript functions, size reduction, time required per page, “running frequency” – which measures how frequently MUZEEL should run depending on how quickly web pages change – and finally both structural and functional similarity between MUZEEL-ed and original pages.

6.2.1 Data Savings. Figures 3(a) and 3(b) show, respectively, the per-JS file distribution of the eliminated number of functions and their corresponding sizes in bytes for the 300,000 crawled JS files. The outer figures show the histograms computed for a total of 100 bins, whereas the inner figures show the Cumulative Distribution Functions (CDFs). The figures show that, on median, MUZEEL eliminates 67 JS functions per file, accounting for about 10 Kbytes. Both distributions show some heavy tails, e.g., with 20% of JS files having between 416 and 34,000 eliminated functions (accounting for between 47 Kbytes and up to 10 Mbytes).

Next, we extend the previous analysis with the number of eliminated JavaScript functions (Figure 3(c)) and bytes (Figure 3(d)) per web page. Figure 3(c) shows a Normal distribution for the per-page number of eliminated JavaScript functions, with mean of about 2,000 functions. For most pages, the number of eliminated JavaScript functions ranges between 100 and 10,000. If we focus on the amount of eliminated bytes (Figure 3(d)), we also observe a Normal distribution with mean of about 700 Kbytes. The figure also shows that most pages benefit from a (JavaScript) size reduction ranging from 10 Kbytes and up to 5Mbytes.

6.2.2 Accuracy. Next, we investigate the accuracy of MUZEEL-ed pages, i.e., their structural and functional similarity with respect to the original pages. The structural similarity refers to visual content completeness between MUZEEL-ed and original pages. We use *QLUE* [28, 29] to decompose screenshots of both page versions into smaller components that are compared against each other using OpenCV’s SSIM [47]. The structural similarity score is computed as the weighted average of the structural similarity of each component, using their respective area as a weight. Instead, the functional similarity score refers to the completeness of the page interactive functionalities between MUZEEL-ed and original pages. We use an interaction-bot to emulate user interactivity, while generating screenshots for each element. We then average the individual similarity of each screenshot to derive the overall functional similarity.

Figure 3(f) shows the CDF of the structural and functional similarity scores for the 200 web pages selected from the larger corpus of 15,000 pages as described in Section 6.1.1. We limit this analysis

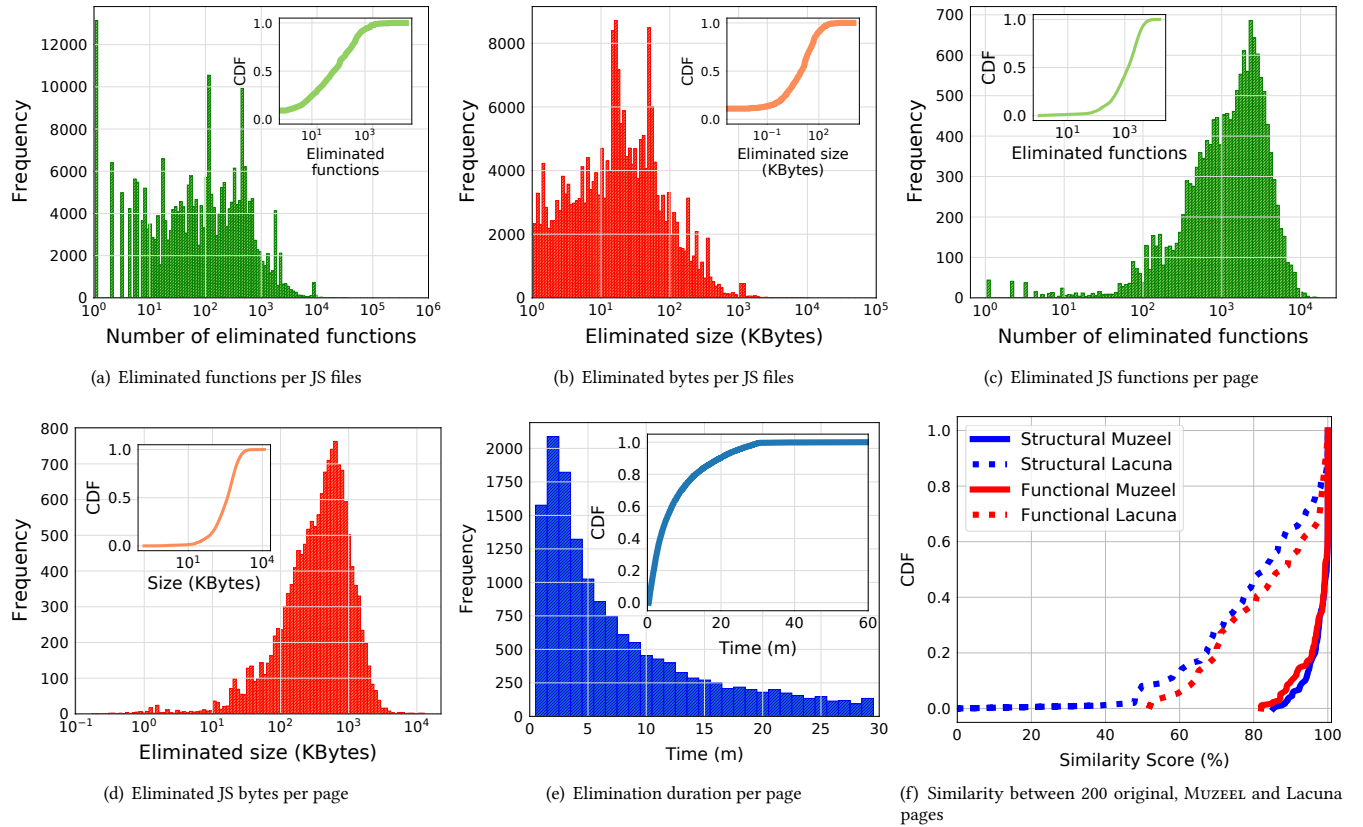


Figure 3: Dead code elimination statistics computed across 15,000 popular web pages

to 200 pages due to the time required to running this tool. We also compare MUZEEL with Lacuna [38], another tool for deadcode elimination. The structural similarity CDF, depicted in solid blue, shows that for 90% of the pages MUZEEL maintains a similarity score of above 90%, in contrast to Lacuna’s 60% similarity score (dashed blue curve). For the lowest scoring 10% pages MUZEEL has achieves a similarity score between 85-90%, unlike Lacuna that has scores as low as 0%. Similar observation can be seen in the functional similarity score of MUZEEL (solid red curve).

6.2.3 Complexity. We evaluate the time required to perform the dead code elimination for each of the 15,000 web pages. Figure 3(e) shows the histogram (and CDF) of the aforementioned time in minutes. The figure shows that MUZEEL requires at most 5 minutes for ~85% of the pages, assuming up to 6 cores used concurrently. For the remaining 15% of the pages, we measured a duration of up to 20 minutes. This result suggests that MUZEEL can easily run each time a web page is updated, to ensure the correctness of the JavaScript dead code elimination. On our server, this process takes a median of 30 seconds when using all 64 cores.

We evaluate MUZEEL’s “running frequency”, or how frequently it should run. We do this by cloning a set of web pages spanning different categories (news, education, sports, business, commercial, entertainment) every 12 hours in a period of a week. Results show

that 99% of the JavaScript after dead code elimination does not change over a period of a week. This means that CDN providers do not need to re-run MUZEEL with frequency less than one week.

6.3 Muzeel Performance

This section evaluates the impact of eliminated JS dead code on user experience. We use 3 Android mobile devices (see Table 1), 4 browsers, 3 network conditions, and the 200 web pages described in Section 6.1.1. We compare the performance of these 200 pages with the dead code eliminated by MUZEEL with respect to their original versions. Each page was loaded 5 times for each version, and for each metric we report the median out of the 5 runs.

6.3.1 Network-Based Evaluation. Figure 4 shows the CDFs of the *delta* performance results of the MUZEEL-ed pages with respect to their original versions, in terms of timings metrics (FirstContentfulPaint, SpeedIndex and PageLoadTime), using both the low-end (Redmi Go) and the high-end (Samsung S10) phones, under three emulated networks: 3G, LTE, and LTE+. For each page and metric, the delta is computed by subtracting the value of that metric measured for the MUZEEL-ed page from the value measured for the corresponding original page. It follows that values bigger than 0 represents MUZEEL savings, while values smaller than zero represent penalties. All these experiments were conducted using Chrome.

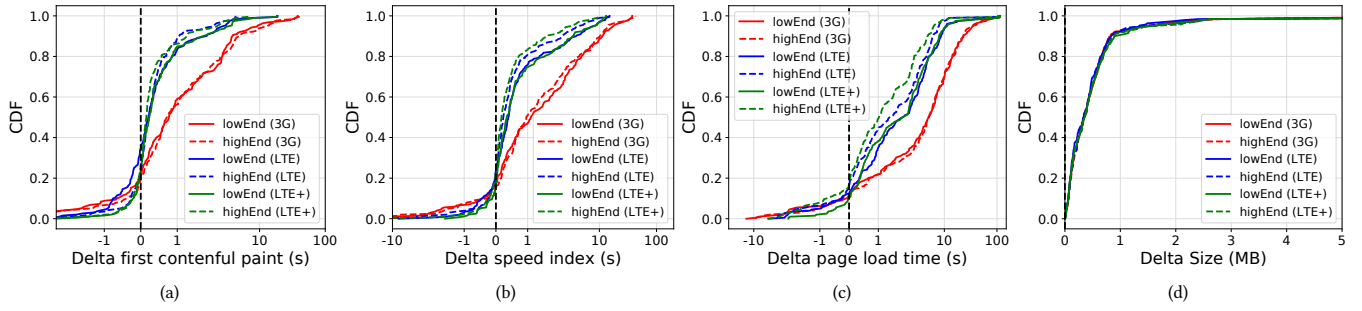
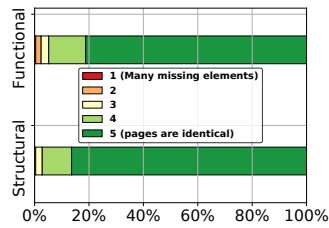
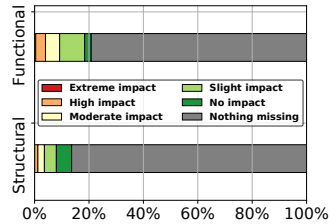


Figure 4: Delta performance results using different networks and phone types.



(a) Rating the similarity of the MUZEEL page to the original (on a scale from 1 to 5)



(b) Rating the impact of missing elements

Figure 5: Muzeel functional and structural similarity and impact of missing elements.

FirstContentFulPaint (FCP) is a web quality metric capturing the first *impression* of a website, which many users often associate with what defines a web page as “fast” [41]. Figure 4(a) shows three trends, regardless of the network condition and the device: some web pages (20–30%) show minimal slow down (average of few hundred ms), some exhibit no performance difference (up to 40% on LTE and LTE+), and the majority (up to 60% in 3G) show significant FCP speedups, up to several seconds. Intuitively, MUZEEL speedups arise from JavaScript which are saved before FCP. Given this metric is quite fast, and JavaScript tend to be loaded later in a page, it is expected to see many web pages with equivalent FCP between their original and MUZEEL-ed version. More unexpected are the few negative results. Their explanation lies in the intricacy of the web—and the HTTP protocol itself—where removing or shrinking some objects can change the order of requests, e.g., by anticipating a larger object even if not contributing to a specific metric. The figure also shows much higher speedups and slowdowns when

considering 3G; this is expected given that the lower bandwidth inflates the differences between the two loading strategies.

Next, we focus on SpeedIndex (SI) a web quality metric which aims at capturing the “average” user experience [41]. Compared with FCP, Figure 4(b) shows a clear shift to the right, with 70–80% of the pages benefiting from speedups. This happens because SI is an overall “later” metric which gives more chances to MUZEEL to offer its savings. The same trend is also confirmed in Figure 4(c), which instead focuses on the PageLoadTime (PLT), or the time at which a browser fires the onLoad event, suggesting that all content has been loaded. In this case, MUZEEL offers speedups for 90–95% of the web pages. With respect to the networking conditions, both figures confirm the previous trend with much higher speedups in 3G. With respect to the type of mobile device, the figures show higher benefit for the low-end device, likely due to a reduction in CPU usage as we will discuss later. Finally, given that the size gain doesn’t depend on the network, the same delta size is shown in Figure 4(d) across multiple phones and networks.

6.3.2 User-Study Evaluation. We conducted a user-study in which participants were asked to compare how structurally and functionally similar MUZEEL-ed pages are in comparison to the original pages. Participants were presented with two browser windows: in the left window, the original page was loaded, and in the right window, the MUZEEL version of the same page was loaded. Participants were asked to navigate both pages and test the functionalities across both. After evaluating each page, a participant fills out a short survey to give their thoughts on the structural and functional similarity of the two versions of the page. Our study consisted of 50 sites, each of which is analyzed by at least three different participants. Participants were recruited from a reputed university via an online job posting board. In total, we received over 390 individual responses from 41 participants. Neither sensitive nor personal information is collected during the study. An institutional review board (IRB) approval is granted to conduct the study, and the authors who conducted the study are CITI [5] certified.

Figure 5(a) shows how participants ranked (1 to 5) the similarity from both a structural and functional perspective. Over 95% of participants rated the structural similarity of each page with a score of 4 or greater and about 95% rated the functional similarity with a score of 4 or greater. Figure 5(b) shows how participants rated the impact of missing structural elements or functionality on their

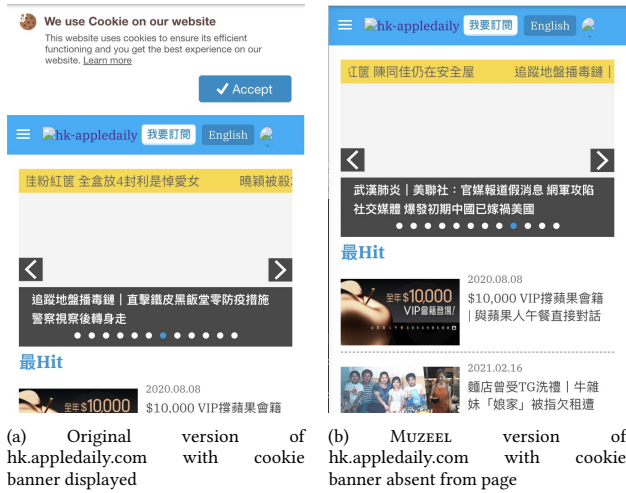


Figure 6: An example showing a page that received a structural similarity score of 3 out of 5 by a user

experience. In terms of functionality, almost 80% of participants reported either no missing functionality or the missing functionality had no impact on their experience. This number rises to 90% when including participants who reported a “slight impact”. For structural elements, almost 90% of the participants reported either no missing elements or missing elements without an impact on their experience. This rises to 95% when responses reported a “slight impact”.

In our user study, the majority of users gave high ratings for both the structural and functional similarity of the MUZEEL-ed pages. We here discuss three examples of lower-rated pages for both the structural and the functional similarity. It’s worth noting that these examples provide representative edge cases in our dataset. The first example is illustrated in Figure 6, which shows a page that received a structural similarity score of 3 by a participant in our study when comparing the MUZEEL version of the page to the original. Figure 6(a) shows a cookie banner that appears at the top of the page, which doesn’t appear in the MUZEEL-ed version of the page (as shown in Figure 6(b)). This is due to a missing event while processing this particular page that causes the function which handles the banner not to be recognized as a used function.

Although MUZEEL considers all the events associated with page elements that are added to the DOM structure during the page load (including the elements hidden by CSS), it fails to consider the elements that are added dynamically to the DOM later, either due to the execution of asynchronous JavaScript or due to the user interactivity. In addition to the banner missing in Figure 6(b), a few button icons are also missing on both pages shown in Figure 6, most likely due to a caching error unrelated to MUZEEL.

Figure 7 shows a second example illustrating a page that received a functional similarity score of 3 by a participant in our study when comparing the MUZEEL version of the page to the original. Figure 7(a) shows that each menu item can be expanded to reveal a sub-menu. While the menu opens successfully in the MUZEEL-ed page, the ability to expand each sub-menu item is a missing

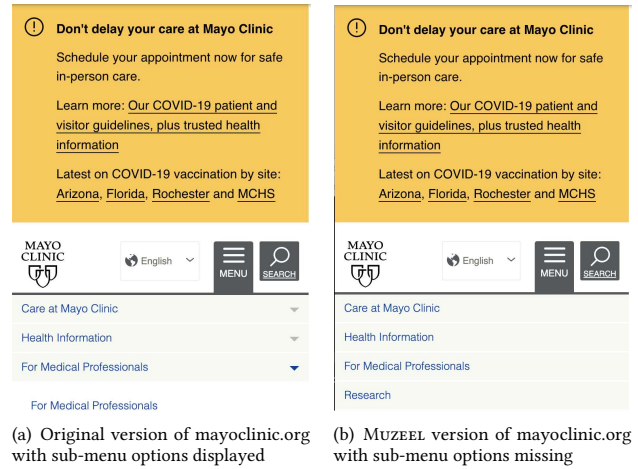


Figure 7: An example showing a page that received a functional similarity score of 3 out of 5 by a user

functionality as shown in Figure 7(b). This is due to the same aforementioned reason that causes the events of each of the sub-menu items not to be recognized during the dead code discovery process. The menu would still be visible to the user in the MUZEEL-ed page, which means that the element representing it was part of the DOM tree when the page is loaded, which was not the case for each of the DOM elements representing the sub-menu items.

In the third example shown in Figure 8, we show a MUZEEL-ed page (Figure 8(b)) that looks exactly the same as the original page, but with a missing functionality. In its original version, the page comes with a BESTSELLERS scrolling functionality which is missing in the MUZEEL-ed version (Figure 8(d) shows that the BESTSELLERS carousel remains unchanged when scrolling over it). The reason for MUZEEL’s failure in this case is tied to an inability to accurately trigger the scroll functionality on the element. This occurs because MUZEEL’s default ‘drag’ implementation doesn’t meet the threshold required to trigger the scroll functionality. This type of error can occur with more complex events, such as ‘drag’ and ‘keydown’ in which the intended functionality of the event can be ambiguous i.e. how far should we drag the element, and which key is expected can be unknown.

6.3.3 Browser-Based Evaluation. We now evaluate MUZEEL across browsers: Chrome, Edge, Firefox, and Brave. We study resource usage (CPU, bandwidth, and battery) and also report on SpeedIndex (SI). Experiments were run both on a high-end (S10) and a low-end device, a Samsung J3 which we can connect to a power meter for fine-grained battery monitoring. We assume the worst case scenario of a fast network (WiFi + fiber) with symmetric download/upload speeds of 100 Mbps, and RTT to the CDN edge of 10 ms.

Figure 9(a) shows the CDF of the delta bandwidth utilization (original - MUZEEL-ed) across browsers. We report only results for the high-end device (S10) since, as expected, they are equivalent to the low-end one (J3). The figure shows equivalent bandwidth savings for MUZEEL among Chrome, Firefox, and Edge: median savings of about 400KB, up to multiple MBytes. When considering

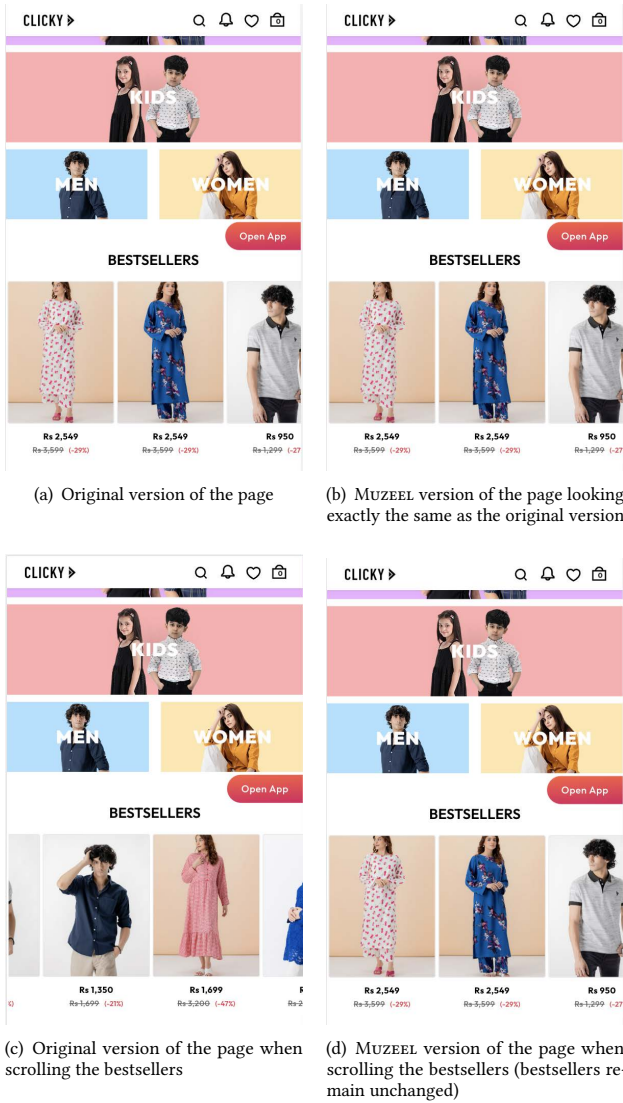


Figure 8: An example showing a MUZEEL-ed page looking exactly the same as the original but with missing functionality

Brave, the bandwidth savings from MUZEEL are reduced by about 50%. This happens because Brave’s ad-blocker remove tracking and advertisement code – mostly JavaScript – thus giving less a chance to MUZEEL to provide savings. Even in the case of Brave, MUZEEL realizes data savings in the order of MB for 10% of the web pages. Note that Brave runs a very aggressive ad-blocker [43], and thus these numbers represent a lower bound on the expected data savings provided by MUZEEL in presence of ad-blocking.

We next investigate how fast (or slow) MUZEEL would make the user experience across browsers and devices. We only report on SpeedIndex (SI) given that it captures the “average” end-user experience and for which the previous section has shown “average” performance improvements. Accordingly, Figure 9(b) shows boxplots (*i.e.*, minimum, first quartile, median, third quartile, and

maximum; outliers are plotted as “o”) SpeedIndex per browser and device. The figure shows an overall trend similar to the previous result (see Figure 4(b)), with about 75% of the websites enjoying performance improvements up to several seconds. Next, 10-20% of the pages (according to browser and device) shows minimal slow down comprised between 200 and 700ms, followed by some outliers of up to 1.2 seconds. The figure shows that Brave benefits from highest speedups; this is likely due to a load reduction on its ad-blocker.

Next, we evaluate MUZEEL’s impact on CPU consumption. We sample CPU usage once per second during a web page load, and then report the median consumption. Figure 9(c) shows boxplots of the (delta) median CPU utilization across browsers and devices. For the low-end device, the figure shows negligible CPU changes (within 1%) for many websites, *e.g.*, between the 25th and 50th percentile for Brave. However, 50% of the websites see significant benefits comprised between 1 and 10% CPU reduction, with some outliers reaching up to 50%. The bottom 25% show some CPU usage increase of up to 10%; such larger CPU degradation is mostly associated with websites with massive speedups (more than one second for 10-20% of the websites) where MUZEEL compresses the overall page load in a shorter time causing a temporary burst in CPU usage. A similar trend is observable for the high-end device (S10), with the caveat that CPU improvements (and penalties) are less pronounced, *e.g.*, the median CPU savings is now close to zero for most browsers.

Finally, we comment on battery consumption – measured via a power meter – on the Samsung J3. Given that mAh are hard to relate to actual savings/penalty, we then report the result as a percentage of the battery consumption of the original version. For about 70% of the pages we observe energy savings ranging from 0 to 30%, regardless of the browser. We measure minimal penalty (up to 8%) of the remainder of the websites; as above, this is correlated with the higher CPU usage related to a much faster user experience.

6.4 MUZEEL Performance on Internal Pages

A common browsing pattern consists of loading a website’s *landing* page, and from there load multiple *internal* pages, *e.g.*, when reading articles reported by a news website. Under such pattern, after loading the landing page, the JavaScript shared between landing and internal pages is loaded from the browser’s cache. This raises the question of how to handle potential functionalities needed by an internal page but not by the landing page. Before discussing technical solutions to handle this approach, we first evaluate the likelihood of such event.

Results – From the previous corpus of webpages, we randomly select four internal pages for each of the 50 landing pages (for a total of 250 unique pages). We then process each page using MUZEEL and compare the code similarity of landing versus internal pages using the k-Shingling algorithm [23], which generates a set of hashed values representing each JavaScript file’s content. We then compare the hashes generated from the landing page against the collection of hashes from the four internal pages: the missing hashes indicate missing functionalities in the internal pages if we were to rely on just running MUZEEL at the landing page.

The results of the comparison show that for 68% of the webpages, the internal pages were completely covered by the MUZEEL-ed JavaScript file produced just by considering the landing page.

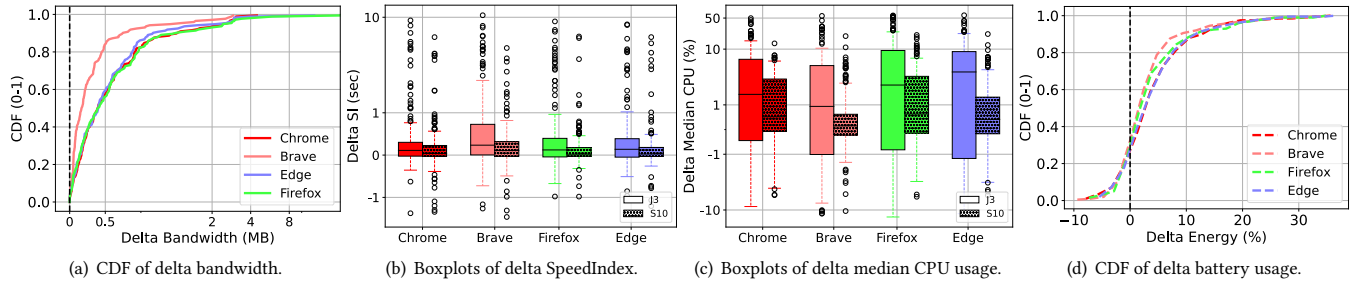


Figure 9: Delta performance results using different browsers and phone types.

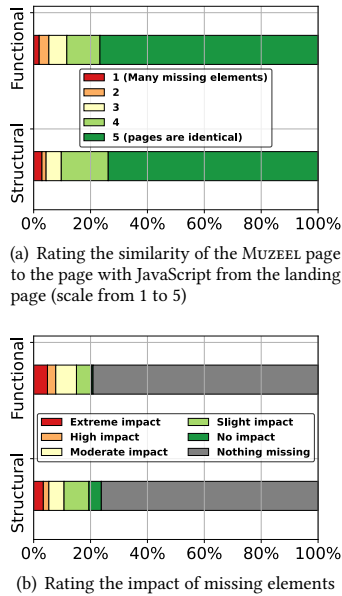


Figure 10: Functional and structural similarity and impact of missing elements when using landing page JavaScript.

This percentage increases to 90% of the webpages if we can tolerate a missing 10% of code. The remainder 10% of pages are more problematic, with between 10 and 40% of code missing (and up to roughly 85% in the worst case). The next logical question is “what is the impact of such missing code on the user experience?”. To answer this question, we performed another user study involving 25 participants, each of which were asked to compare two versions of 10 different web pages and provide a score out of 10. In a left window, an internal page was loaded with its own JavaScript files after being processed by MUZEEL, while in a right window, the same page was loaded with JavaScript files created by MUZEEL using the landing page as a reference. Participants were then asked to compare each couple of versions against each other in terms of content similarity as well as the impact of missing elements.

Figure 10(a) shows that over 85% of participants judged that the JavaScript from the landing page yielded a functional similarity rating of 4 or higher. In terms of structural similarity, about 90% of

participants gave a rating of 4 or higher. Figure 10(b) shows that for about 80% of the pages the participants felt that their experience on these pages was not impacted in any way when considering both structural and functional components of the page.

Discussion. The previous analysis indicates that the majority of internal pages can simply rely on the MUZEEL-ed JavaScript content produced from the landing page. In fact, either all their functionalities are covered or a small portion is missing with minimal impact on the user experience. However, if a CDN provider is willing to run MUZEEL per page, it is always possible to perform the above analysis, identify the missing JavaScript functionalities, and add them to the internal page (either statically or dynamically). This approach ensures minimal missing functionality and integrates well with browser caching. The side effect here is the extra cost required to run MUZEEL per page. However, operators may choose not to use MUZEEL due to the possibility of breaking the page functionality.

6.5 MUZEEL In the Wild

We conclude our evaluation by deploying MUZEEL in the wild. To do so, we deployed 2 low-end phones (Redmi Go) in UAE (Abu Dhabi) and Ireland (Tralee), and one high-end phone (Samsung S10) in US-EAST (New Jersey) connected via three mobile operators: Etisalat (UAE), Vodafone (Ireland), and Mint (US-EST). We further extended our CDN setup using as edge nodes two AWS instances (US-EAST and Ireland) and one cloud node from [1] in UAE (due to the current lack of AWS). We then repeat the experiments described in the previous section, using the same set of pages and limiting to Google Chrome as a browser. We run experiments both on 4G and 3G. Results are summarized in Figure 11.

Overall, results for data and performance savings confirm, in the wild, the results obtained via network emulation. Most notably, performance improvements are more remarkable on lower-end phones and slower network, e.g., an extra 500 ms SpeedIndex improvement on average (see Figure 11(b)) when comparing results from Mint 4G and Vodafone 3G.

7 CONCLUSION

Over the years, web pages have been growing in complexity offering more functionalities to users. This growth has been fueled by JavaScript, the key programming language to make web pages interactive. Anecdotal evidence indicates that lots of this JavaScript code only helps in bloating web pages, since many of its functions

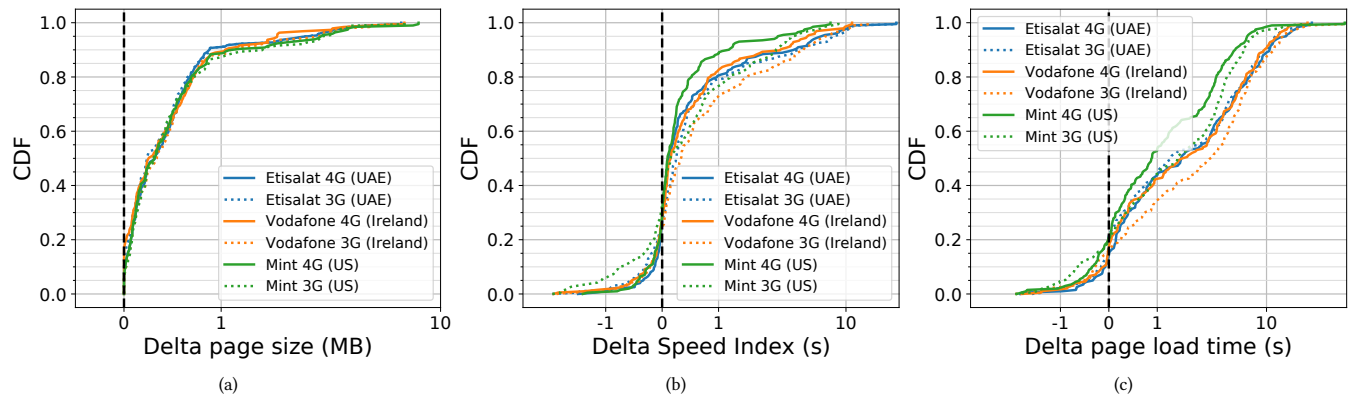


Figure 11: Delta performance results of MUZEEL in the wild over different ISPs and countries.

are never called. This is because web developers resort to large and popular JavaScript libraries even for simple tasks, thus only using a small portion of the code they import. The elimination of such *dead* JavaScript code has potential to offer great performance savings (data, CPU, memory, and battery) to web users. However, the identification of dead JavaScript code is hard since it requires either deep knowledge of the code, which is normally lacking, or extensive and invasive user traces. In this paper we propose MUZEEL, a black-box approach to JavaScript dead code identification. MUZEEL’s fundamental idea is to explore all the possible states a web page can reach via emulation of user interaction and browser event tracking. We test MUZEEL against 15,000 web pages and demonstrate high accuracy (average structural similarity score of 98%, and 97% for functional similarity) and speed (~ 7.5 minutes per web page on a modern desktop machine). This analysis further reveals that half of the JS files have at least 70% of unused functions (accounting for an extra 400KB, on average), providing concrete data behind the anecdotal evidence which motivated this and other related work. Last but not least, MUZEEL shows the ability to speedup webpages by several seconds on low-end phones and connections (3G), which is paramount in the fight to the digital divide problem.

REFERENCES

- [1] 2005. ae Servers: Dedicated and cloud servers in Dubai data centers. <https://www.aeserver.com/vps-hosting.php>. (2005). Accessed: 2022-02-02.
- [2] 2009. jsoup: Java HTML Parser. <https://jsoup.org/>. (2009). Accessed: 2021-01-19.
- [3] 2015. Hypertext Transfer Protocol Version 2 (HTTP/2). <https://datatracker.ietf.org/doc/html/rfc7540>. (2015).
- [4] 2017. XML Path Language (XPath) 3.1. (2017). <https://www.w3.org/TR/2017/REC-xpath-31-20170321/>
- [5] 2019. CITI Program - Collaborative Institutional Training Initiative. www.citiprogram.org. (2019). Accessed: 2019-10-10.
- [6] 2020. Which is the Best Free Ad Blocker? <https://brave.com/learn/best-ad-blocker/>. (2020). Accessed: 2021-05-26.
- [7] 2021. 2020 state of the CDN industry. <https://blog.intracately.com/2020-state-of-the-cdn-industry-trends-market-share-customer-size>. (2021). Accessed: 2021-03-19.
- [8] 2021. Android Debug Bridge (adb). <https://developer.android.com/studio/command-line/adb>. (2021). Accessed: 2021-05-26.
- [9] 2021. Brave Passes 25 Million Monthly Active Users. <https://brave.com/25m-mau/>. (2021). Accessed: 2021-05-26.
- [10] 2021. Calculate visual performance metrics from a video. <https://github.com/WPO-Foundation/visualmetrics>. (2021).
- [11] 2021. Chrome DevTools Protocol. (2021). <https://chromedevtools.github.io/devtools-protocol/tot/DOMDebugger/#method-getEventListeners>
- [12] 2021. A Free and Open Source Interactive HTTPS Proxy. <https://mitmproxy.org/>. (2021). Accessed: 2021-04-28.
- [13] 2021. The State of Mobile Internet Connectivity 2021. <https://www.gsma.com/r/wp-content/uploads/2021/09/The-State-of-Mobile-Internet-Connectivity-Report-2021.pdf>. (2021). Accessed: 2021-12-30.
- [14] 4g.co.uk. 2021. How fast are 4G and 5G? <https://www.4g.co.uk/how-fast-is-4g/>. (2021). Accessed: 2020-09-06.
- [15] Akamai. 2018. Experiment: Improving Page Load Times with Script-Streaming. <https://developer.akamai.com/blog/2018/07/17/experiment-improving-page-load-times-script-streaming>. (2018).
- [16] Akamai. 2020. RUM JavaScript. <https://learn.akamai.com/en-us/webhelp/ion/real-user-monitoring-guide/GUID-E6BE104B-2EB6-463B-A0C8-7A3D0E746446.html>. (2020).
- [17] Alexa. 2020. <https://www.alexa.com/topsites>. (2020).
- [18] Mihai Bazon. 2012. UglifyJS. <http://lisperator.net/uglifyjs/>. (2012). Accessed: 2020-05-01.
- [19] bytesDigital. 2021. The Ultimate Guide to 5G. <https://www.bytesdigital.co.uk/guides/the-ultimate-guide-to-5g>. (2021). Accessed: 2020-09-06.
- [20] Moumena Chaqfeh, Russell Coke, Jacinta Hu, Waleed Hashmi, Lakshmi Subramanian, Talal Rahwan, and Yasir Zaki. 2022. JSAnalyzer: A Web Developer Tool for Simplifying Mobile Web Pages Through Non-Critical JavaScript Elimination. *ACM Transactions on the Web (TWEB)* (2022).
- [21] Moumena Chaqfeh, Muhammad Haseeb, Waleed Hashmi, Patrick Inshuti, Manesha Ramesh, Matteo Varvello, Fareed Zaffar, Lakshmi Subramanian, and Yasir Zaki. 2021. To Block or Not to Block: Accelerating Mobile Web Pages On-The-Fly Through JavaScript Classification. *arXiv preprint arXiv:2106.13764*.
- [22] Moumena Chaqfeh, Yasir Zaki, Jacinta Hu, and Lakshmi Subramanian. 2020. JSCleaner: De-Cluttering Mobile Websites Through JavaScript Cleanup. In *Proceedings of The Web Conference 2020*. 763–773.
- [23] D Manning Christopher, Raghavan Prabhakar, and SCHUTZE Hinrich. 2008. Introduction to information retrieval. (2008).
- [24] CircleCell. 2011. JSCompress - The JavaScript Compression Tool. <https://jscompress.com/>. (2011). Accessed: 2020-05-01.
- [25] Gion Kunz. 2019. CHARTIST.JS SIMPLE RESPONSIVE CHARTS. <https://gionkunz.github.io/chartist-js/>. (2019).
- [26] Utkarsh Goel and Moritz Steiner. 2020. System to Identify and Elide Superfluous JavaScript Code for Faster Webpage Loads. *arXiv preprint arXiv:2003.07396* (2020).
- [27] Utkarsh Goel and Moritz Steiner. 2020. System to Identify and Elide Superfluous JavaScript Code for Faster Webpage Loads. *arXiv preprint arXiv:2003.07396* (2020).
- [28] Waleed Hashmi, Moumena Chaqfeh, Lakshmi Subramanian, and Yasir Zaki. 2020. PQual: Automating Web Pages Qualitative Evaluation. In *The Adjunct Publication of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. Association for Computing Machinery, New York, NY, USA.
- [29] Waleed Hashmi, Moumena Chaqfeh, Lakshminarayanan Subramanian, and Yasir Zaki. 2022. QJUE: A Computer Vision Tool for Uniform Qualitative Evaluation of Web Pages. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*. Association for Computing Machinery, New York, NY, USA, 2400–2410. <https://doi.org/10.1145/3485447.3512112>
- [30] Jason Huggins. 2019. Selenium WebDriver. Browser Automation. <https://www.seleniumhq.org/projects/webdriver/>. (2019). Accessed: 2019-05-14.
- [31] Marja Hölttä and Daniel Vogelheim. 2015. New JavaScript techniques for rapid page loads. <https://blog.chromium.org/2015/03/new-javascript-techniques-for-rapid.html>. (2015). Accessed: 2021-05-4.

- [32] Kayce Basques. 2018. Find Unused JavaScript And CSS With The Coverage Tab. <https://developer.chrome.com/docs/devtools/coverage/>. (2018).
- [33] Masood Mansoori, Yuichi Hirose, Ian Welch, and Kim-Kwang Raymond Choo. 2016. Empirical analysis of impact of HTTP referer on malicious website behaviour and delivery. In *2016 IEEE 30th International Conference on Advanced Information Networking and Applications (AINA)*. IEEE, 941–948.
- [34] Mozilla. 2021. console.log(). <https://developer.mozilla.org/en-US/docs/Web/API/Console/log>. (2021).
- [35] Usama Naseer, Theophilus A Benson, and Ravi Netravali. 2021. WebMedic: Disentangling the Memory-Functionality Tension for the Next Billion Mobile Web Users. In *Proceedings of the 22nd International Workshop on Mobile Computing Systems and Applications*. 71–77.
- [36] Ravi Netravali, Ameesh Goyal, James Mickens, and Hari Balakrishnan. 2016. Polaris: Faster Page Loads Using Fine-grained Dependency Tracking. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. USENIX Association, Santa Clara, CA. <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/netravali>
- [37] Ravi Netravali and James Mickens. 2018. Prophecy: Accelerating Mobile Page Loads Using Final-state Write Logs. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 249–266. <https://www.usenix.org/conference/nsdi18/presentation/netravali-prophecy>
- [38] Niels Groot Obbink, Ivano Malavolta, Gian Luca Scoccia, and Patricia Lago. 2018. An extensible approach for taming the challenges of JavaScript dead code elimination. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 291–401.
- [39] phantomjs. 2018. PhantomJS - Scriptable Headless Browser. <https://phantomjs.org/>. (2018).
- [40] Vaspil Ruamviboonsuk, Ravi Netravali, Muhammed Uluyol, and Harsha V Madhyastha. 2017. Vroom: Accelerating the mobile web with server-aided dependency resolution. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 390–403.
- [41] Karolina Szczur. 2020. Performance | 2020 | The Web Almanac by HTTP Archive. <https://almanac.httparchive.org/en/2020/performance>. (2020). Accessed: 2021-05-26.
- [42] Matteo Varvello, Kleomenis Katevas, Wei Hang, Mihai Plesa, Hamed Haddadi, Fabián E Bustamante, and Benjamin Livshits. 2019. BatteryLab, a distributed power monitoring platform for mobile devices: demo abstract. In *Proceedings of the 17th Conference on Embedded Networked Sensor Systems*. 386–387.
- [43] Matteo Varvello and Benjamin Livshits. 2021. Shedding (Some) Light on Mobile Browsers Energy Consumption. In *TMA*.
- [44] Hernán Ceferino Vázquez, Alexandre Bergel, S Vidal, JA Díaz Pace, and Claudia Marcos. 2019. Slimming javascript applications: An approach for removing unused functions from javascript libraries. *Information and Software Technology* 107 (2019), 18–29.
- [45] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. 2013. Demystifying Page Load Performance with WProf. In *Presented as part of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. USENIX, Lombard, IL, 473–485. https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/wang_xiao
- [46] Xiao Sophia Wang, Arvind Krishnamurthy, and David Wetherall. 2016. Speeding up Web Page Loads with Shandian. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. USENIX Association, Santa Clara, CA, 109–122. <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/wang>
- [47] Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612. <https://doi.org/10.1109/TIP.2003.819861>
- [48] WPO-Foundation. 2017. WebPageTest connectivity settings. <https://github.com/WPO-Foundation/webpagetest/blob/master/www/settings/connectivity.ini.sample>. (2017). Accessed: 2020-09-06.